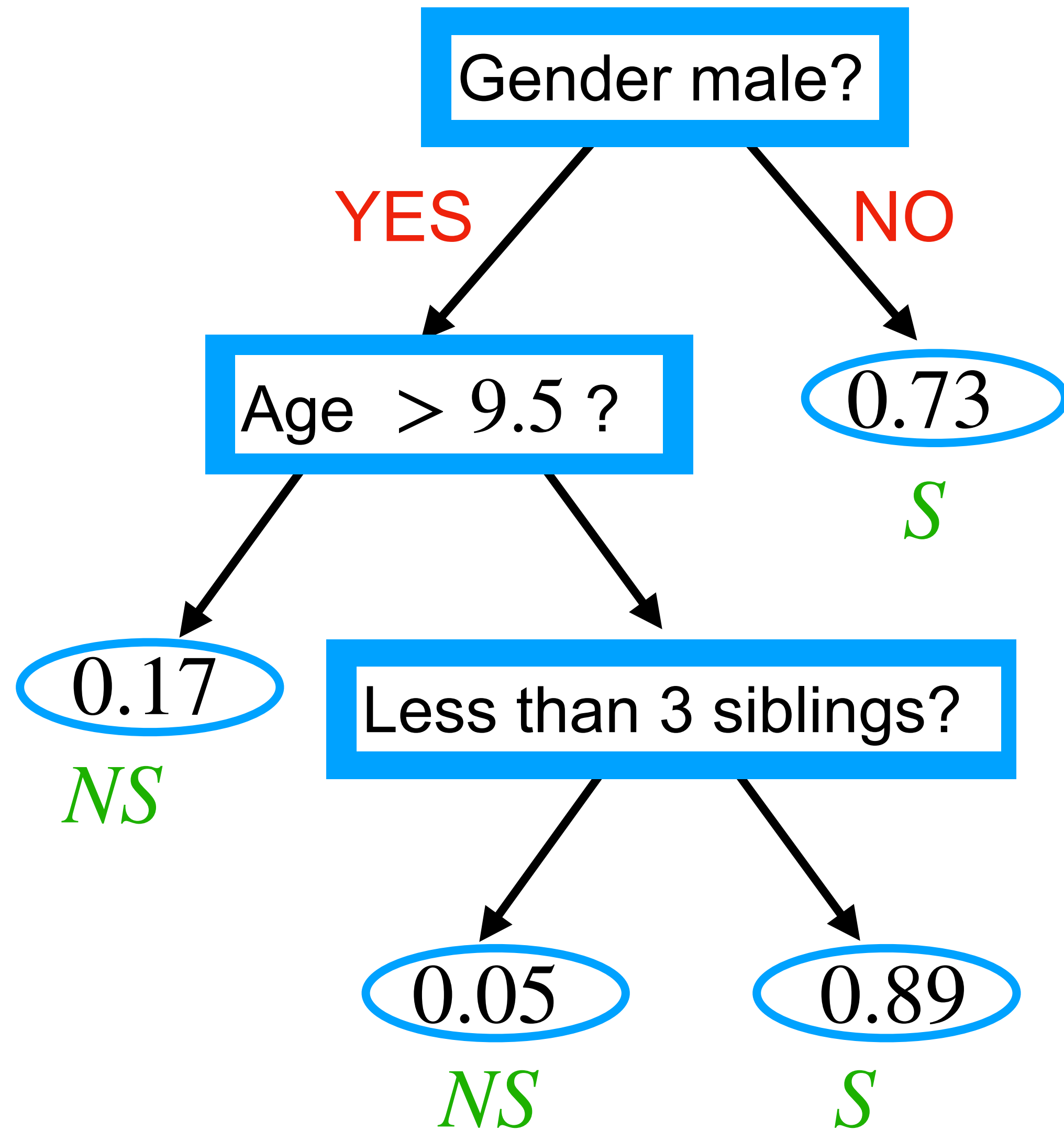


QUIZ

What is the definition of 'margin' in support vector machines?

REVIEW OF PREVIOUS LECTURE

Decision tree:



For classifying a new test point:

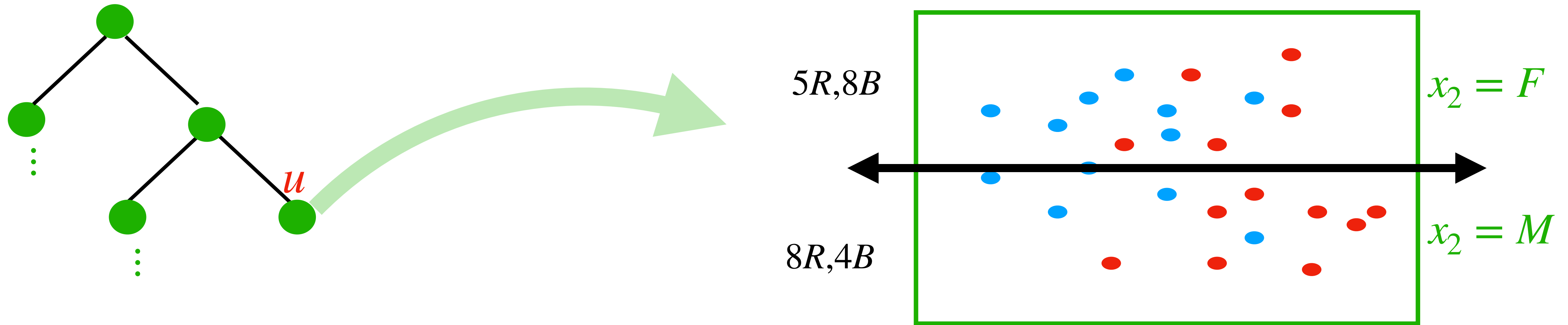
Start at the root of the tree.

Traverse the tree until a terminal node.

Classify with the majority class.

REVIEW OF PREVIOUS LECTURE

Top-down, greedy strategy to construct the tree:



Quality of the split:

Alternative-1: Gini index (impurity) of a set of training data points P ,

$$G(P) = 1 - \sum_{j=1}^K p_j^2$$

Find split minimizing weighted average of $G(P_1)$, $G(P_2)$, where P_1, P_2 are partitions after the split.

HOW IS A DECISION TREE CONSTRUCTED?

How to measure quality of split?

Alternative 2:

$$\textit{Entropy} = D = - \sum_{j=1}^K p_j \log p_j$$

K : number of classes

p_j : proportion of training data points from class j .

HOW IS A DECISION TREE CONSTRUCTED?

How to measure quality of split?

Alternative 2:

$$\textit{Entropy} = D = - \sum_{j=1}^K p_j \log p_j$$

K : number of classes

p_j : proportion of training data points from class j .

Intuition: Same as Gini index

Good split: Minimizes **Entropy of the split**

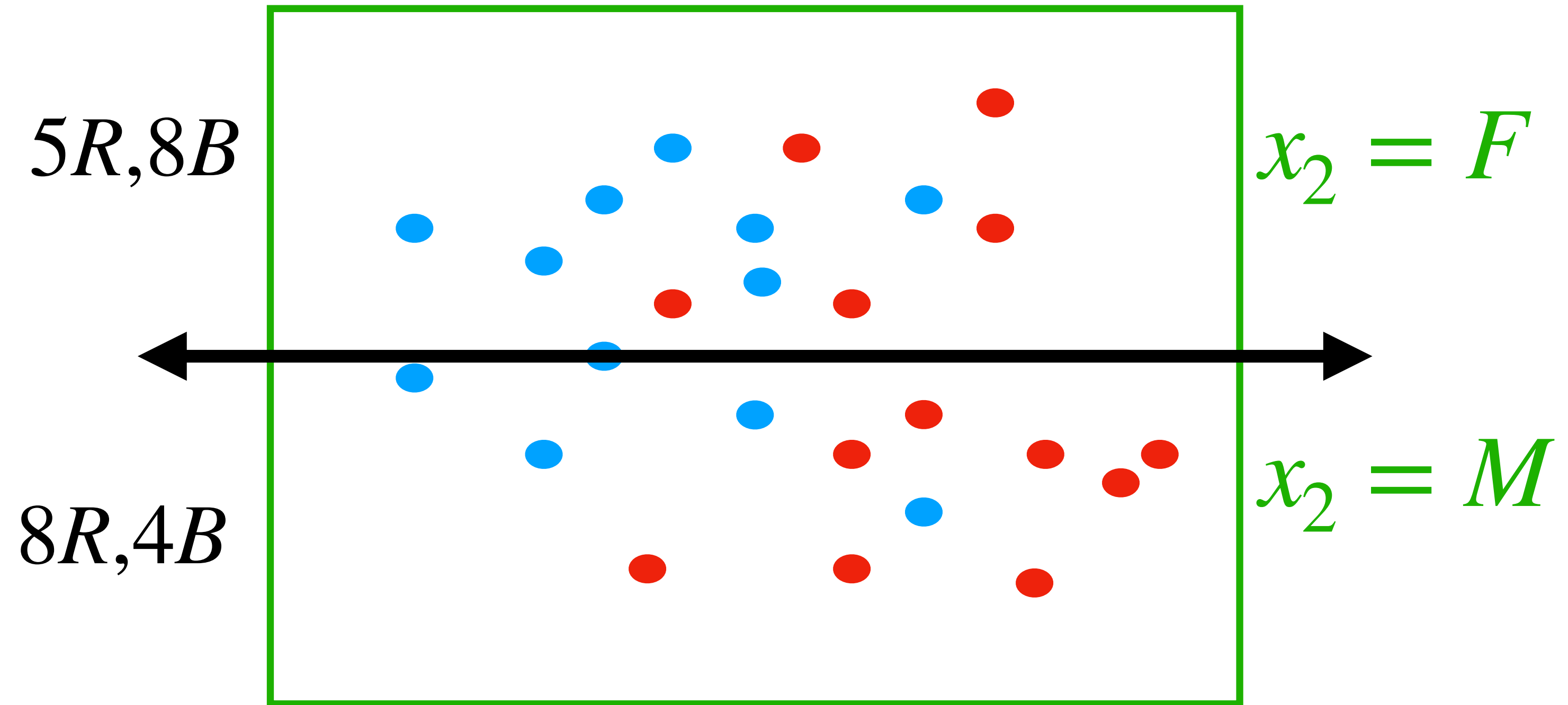
Weighted-sum of Entropy of the two partitions



Proportion of data points in the partition.

HOW IS A DECISION TREE CONSTRUCTED?

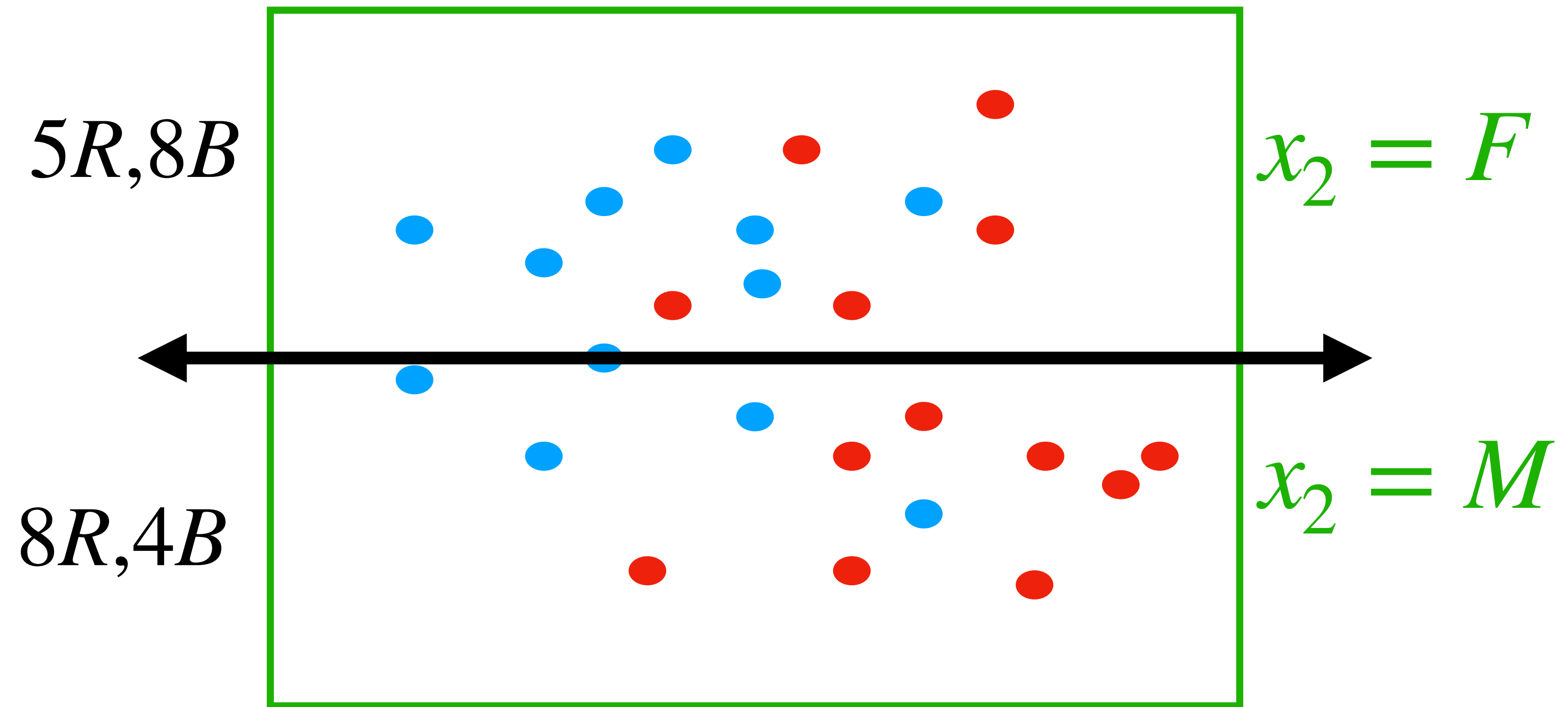
How to measure quality of split?



Example: What is the Entropy of the split?

HOW IS A DECISION TREE CONSTRUCTED?

How to measure quality of split?



Example: What is the Entropy of the split?

$$-\frac{13}{25} \left[\frac{5}{13} \log \frac{5}{13} + \frac{8}{13} \log \frac{8}{13} \right] - \frac{12}{25} \left[\frac{8}{12} \log \frac{8}{12} + \frac{4}{12} \log \frac{4}{12} \right]$$

HOW TO IMPROVE PREDICTIVE PERFORMANCE?

The decision tree is prone to overfitting.

⇒ Classification accuracy of new items can be poor.

Techniques to improve:

- Bagging (**B**ootstrap **A**ggregation)
- Random forests
- Boosting

BAGGING

General purpose method for reducing variance of a learning method.

Background:

Recall that n independent observations each with variance $\sigma^2 \Rightarrow$
variance of sample mean σ^2/n .

BAGGING

General purpose method for reducing variance of a learning method.

Background:

Recall that n independent observations each with variance $\sigma^2 \Rightarrow$ variance of sample mean σ^2/n .

If we had access to **n independent training sets**, everything would be simple. In practice we don't. Use next best thing, bootstrapping.

BAGGING

General purpose method for reducing variance of a learning method.

Background:

Recall that n independent observations each with variance $\sigma^2 \Rightarrow$ variance of sample mean σ^2/n .

If we had access to n independent training sets, everything would be simple. In practice we don't. Use next best thing, bootstrapping.

Bootstrapping: Generate B training sets by sampling with replacement from the training set at hand.

- Train B trees on bootstrapped training sets.
- Take the majority vote for making a prediction.

RANDOM FOREST

Similar to bagging:

Build a number of decision trees on **bootstrapped** training samples.

This time when building trees:

Deciding on a split: **don't use all m features.**

Split choice is on **a random selection** of p predictors.

(Typically $p = \sqrt{m}$)

A fresh selection of random features is done at each split.

Making a random selection of predictors '**decorrelates**' the trees.

RANDOM FOREST

```
class sklearn.ensemble.RandomForestClassifier(n_estimators=100, *, criterion='gini',  
max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0,  
max_features='sqrt', max_leaf_nodes=None, min_impurity_decrease=0.0, bootstrap=True,  
oob_score=False, n_jobs=None, random_state=None, verbose=0, warm_start=False,  
class_weight=None, ccp_alpha=0.0, max_samples=None, monotonic_cst=None)
```

Number of trees.

Number of samples to draw.

The whole dataset or bootstrapped training datasets.

BOOSTING

When we employ ensemble learning (like multiple trees etc.)

vote count by **majority vote**: every vote contributes equally

Problem:

Item/voter	V_1	V_2	V_3	V_4	V_5	Majority
A	*		*	*	*	*
B	*		*	*	*	*
C	*	*		*	*	*
D	*	*				
E		*	*			
% correct	80%	60%	60%	60%	60%	60%

* indicates a correct classification by a voter. Think of each voter as a decision tree.

BOOSTING

When we employ ensemble learning (like multiple trees etc.)
vote count by **majority vote**: every vote contributes equally

Problem: Voters not getting hard items right.

Item/voter	V_1	V_2	V_3	V_4	V_5	Majority
A	*		*	*	*	*
B	*		*	*	*	*
C	*	*		*	*	*
D	*	*				
E		*	*			
% correct	80%	60%	60%	60%	60%	60%

BOOSTING

When we employ ensemble learning (like multiple trees etc.)
vote count by **majority vote**: every vote contributes equally

Problem: Voters not getting hard items right.

Item/voter	V_1	V_2	V_3	V_4	V_5	Majority
A	*		*	*	*	*
B	*		*	*	*	*
C	*	*		*	*	*
D	*	*				
E		*	*			
% correct	80%	60%	60%	60%	60%	60%

Items hard to classify

Large weights to voters classifying hard items right $\Rightarrow$ better classifier.

BOOSTING

When we employ ensemble learning (like multiple trees etc.)

vote count by **majority vote**: every vote contributes equally

Problem:

Item/voter	V_1	V_2	V_3	V_4	V_5	Majority	Best weights
A	*		*	*	*	*	*
B	*		*	*	*	*	*
C	*	*		*	*	*	*
D	*	*					*
E		*	*				*
% correct	80%	60%	60%	60%	60%	60%	100%
best weight	1/3	1/3	1/3	0	0		

Large weights to voters classifying hard items right $\Rightarrow$ better classifier.

BOOSTING

When we employ ensemble learning (like multiple trees etc.)

vote count by **majority vote**: every vote contributes equally

How to assign weights to

Items based on difficulty to classify

Decision trees based on success in classifying large weight items

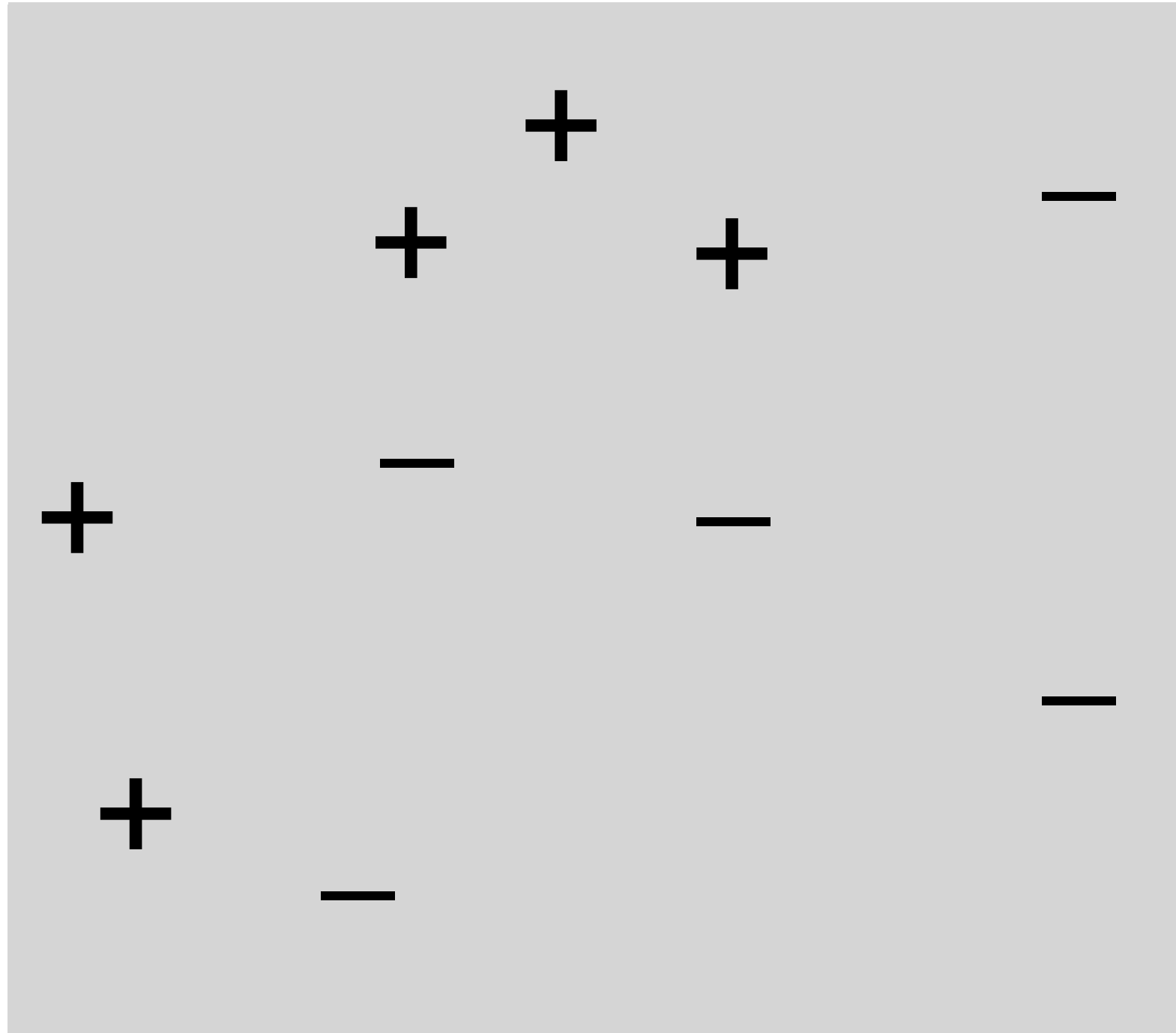
A BOOSTING ALGORITHM: AdaBoost

For t in $1 \dots T$:

Find weight of decision tree and
update the weights of items iteratively.

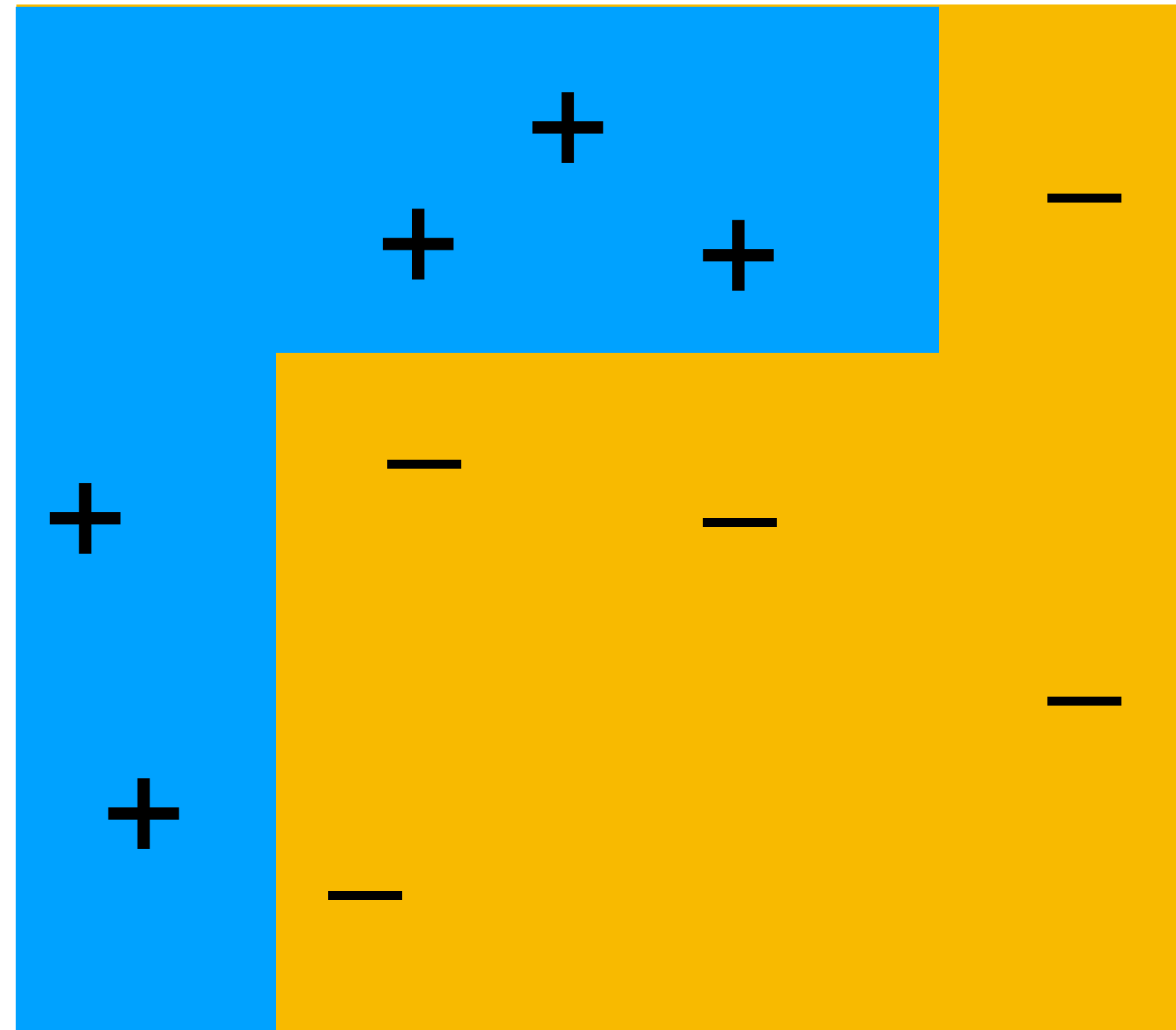
- Choose $f_t(x)$:
 - Find weak learner $h_t(x)$ that minimizes ϵ_t , the weighted sum error for misclassified points $\epsilon_t = \sum_i w_{i,t}$
 - Choose $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$
- Add to ensemble:
 - $F_t(x) = F_{t-1}(x) + \alpha_t h_t(x)$
- Update weights:
 - $w_{i,t+1} = (w_{i,t})e^{-y_i \alpha_t h_t(x_i)}$ for all i
 - Renormalize $w_{i,t+1}$ such that $\sum_i w_{i,t+1} = 1$

A BOOSTING ALGORITHM: AdaBoost



Construct simple decision trees
each with only one internal node!

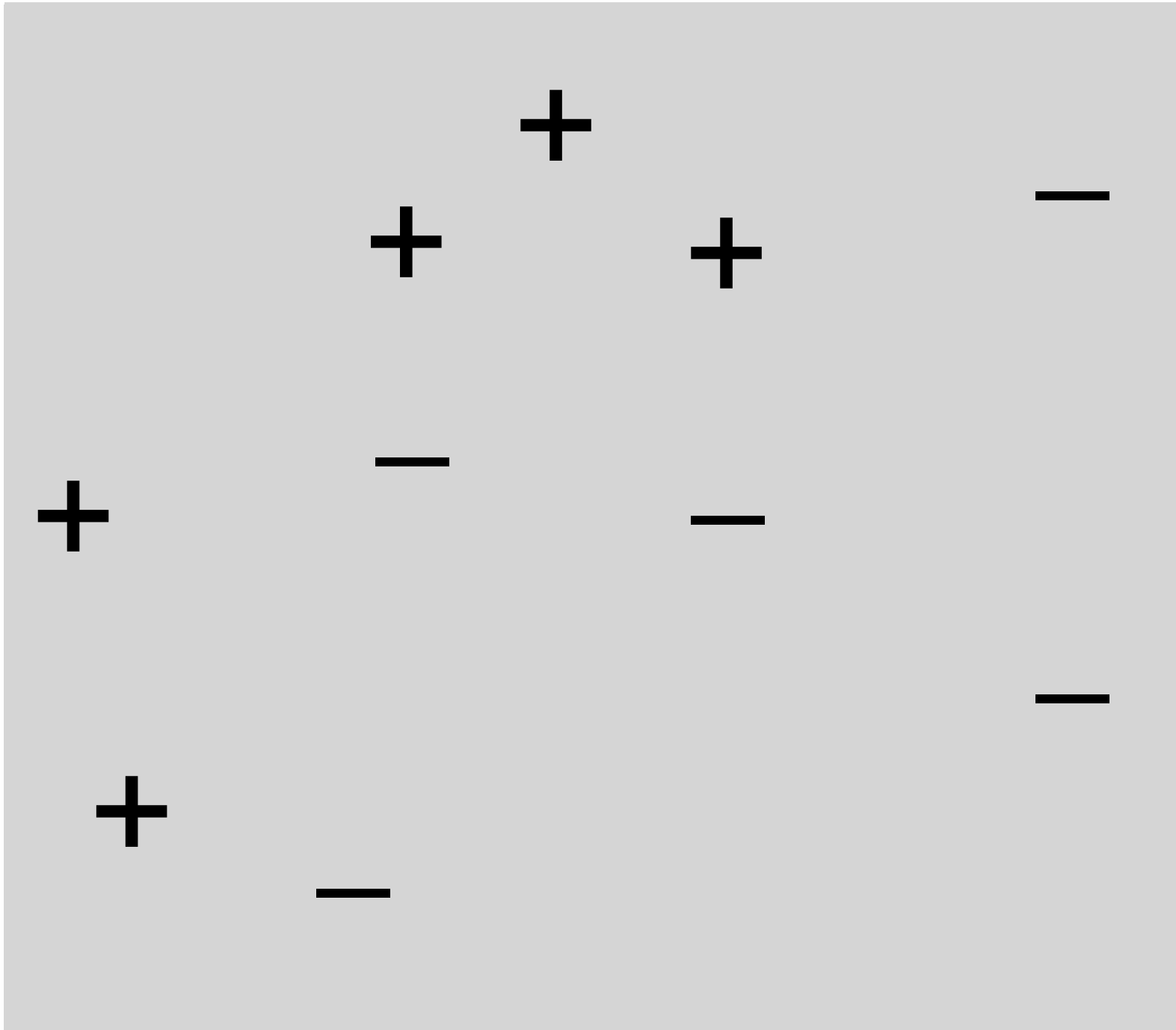
A BOOSTING ALGORITHM: AdaBoost



Desired
classifier

Construct simple decision trees
each with only one internal node!

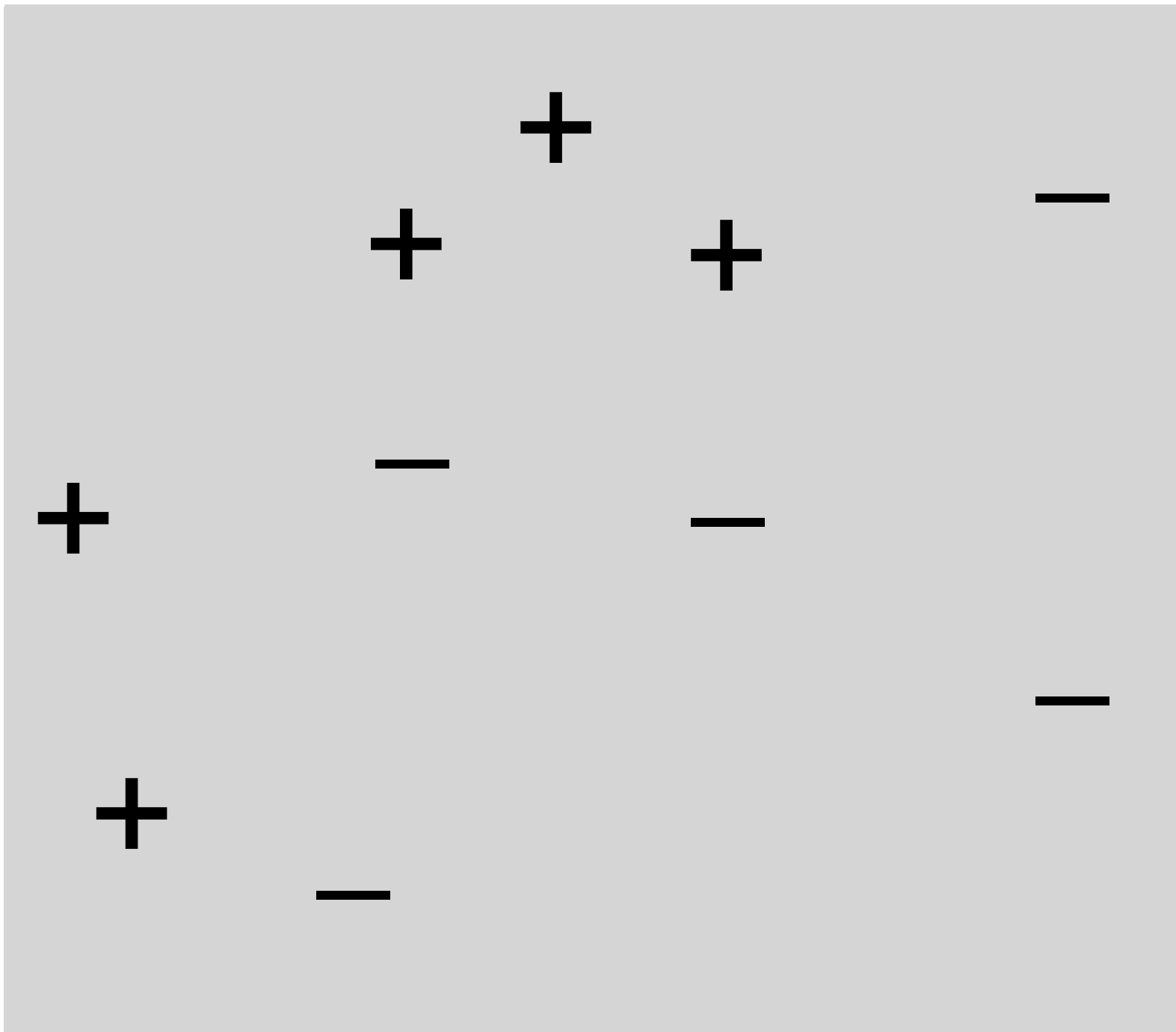
A BOOSTING ALGORITHM: AdaBoost



1	5	+
2	3	+
3	2	-
4	6	-
4	7	+
5	9	+
6	5	-
6	7	+
8	5	-
8	8	-

Training data

A BOOSTING ALGORITHM: AdaBoost



1	5	+
2	3	+
3	2	-
4	6	-
4	7	+
5	9	+
6	5	-
6	7	+
8	5	-
8	8	-

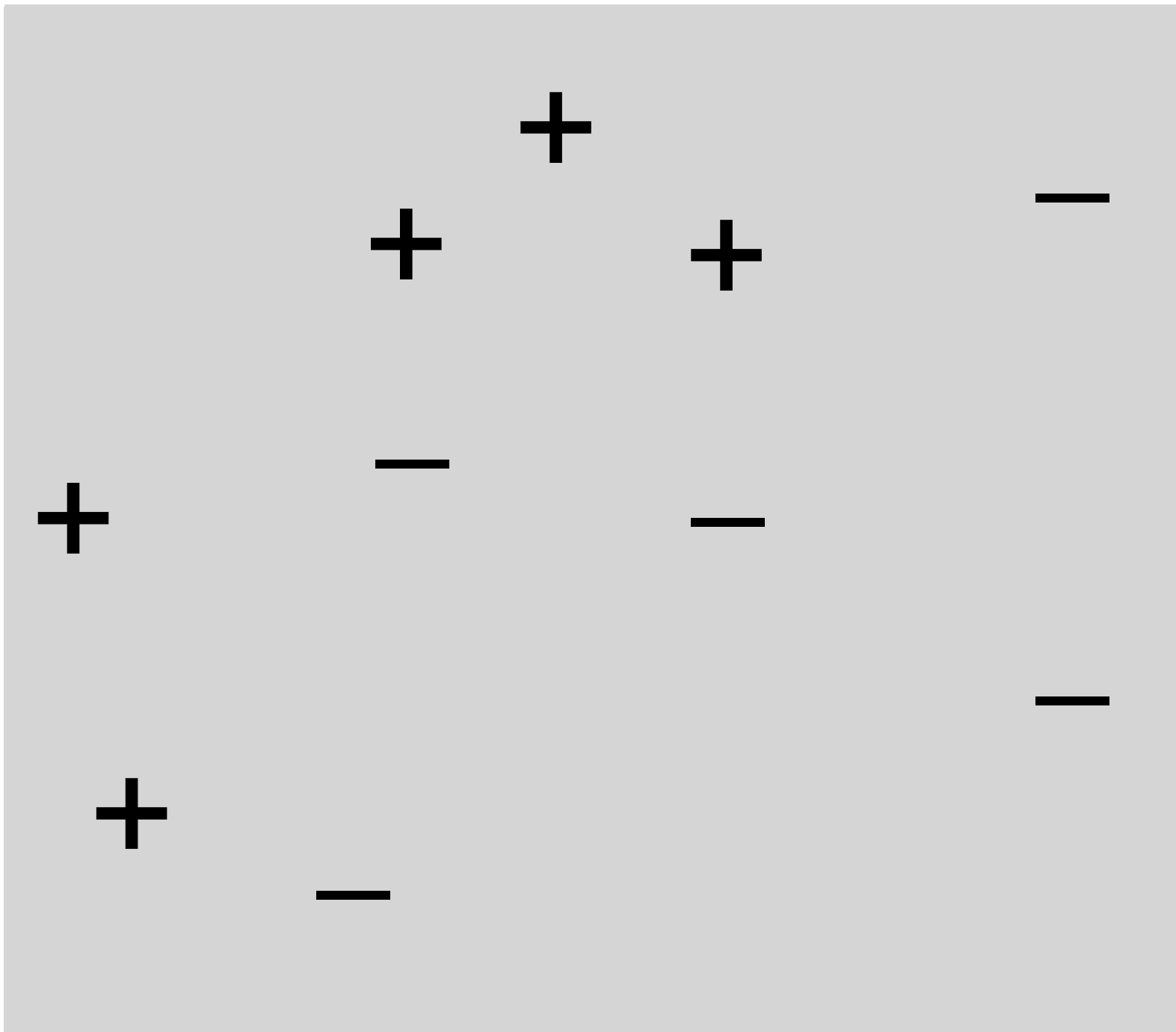
Training data

0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

Initially
 $1/n$

A BOOSTING ALGORITHM: AdaBoost



1	5	+
2	3	+
3	2	-
4	6	-
4	7	+
5	9	+
6	5	-
6	7	+
8	5	-
8	8	-

Training data

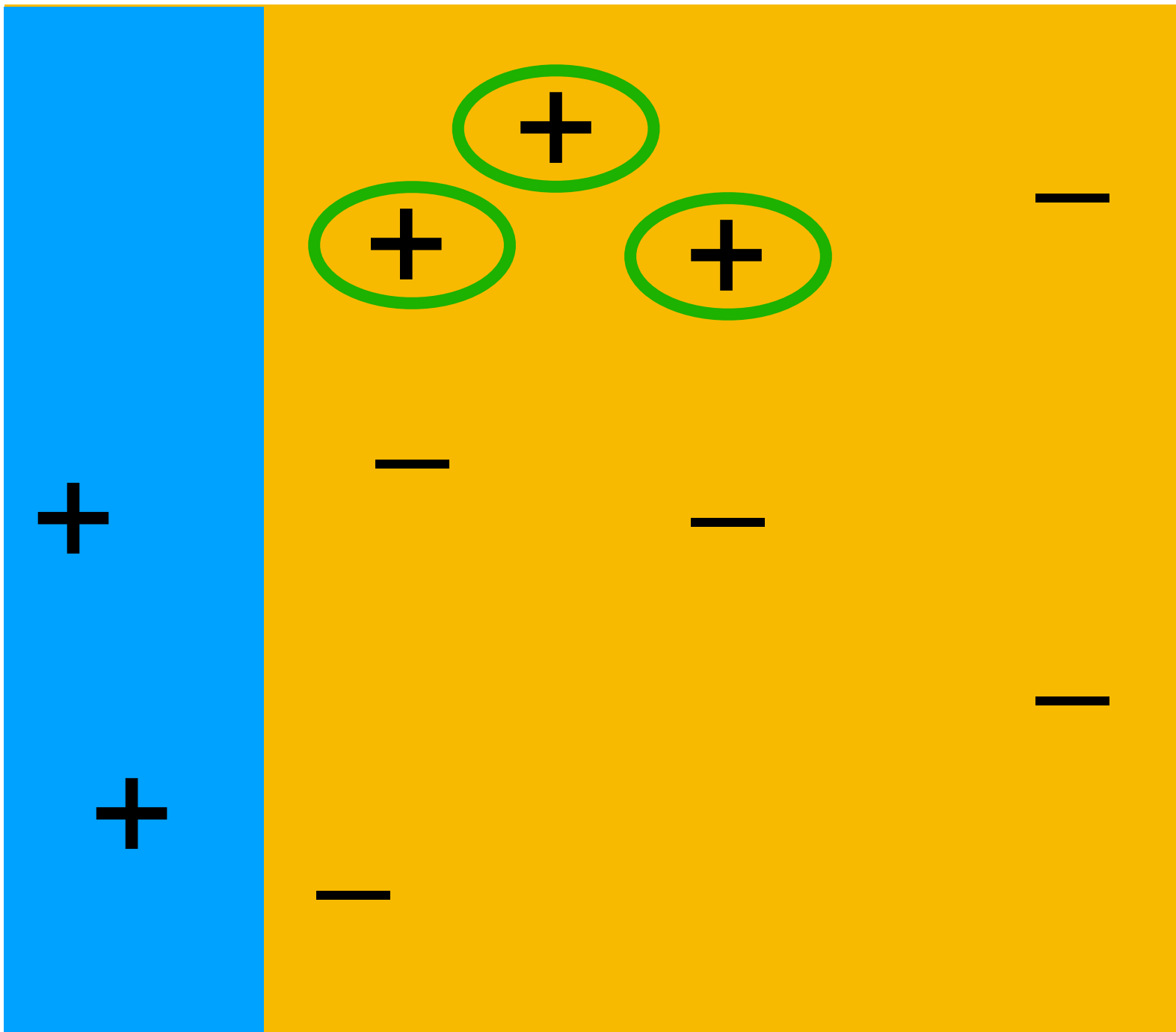
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

Initially
 $1/n$

What is the decision tree with one internal node that minimizes classification error?

A BOOSTING ALGORITHM: AdaBoost



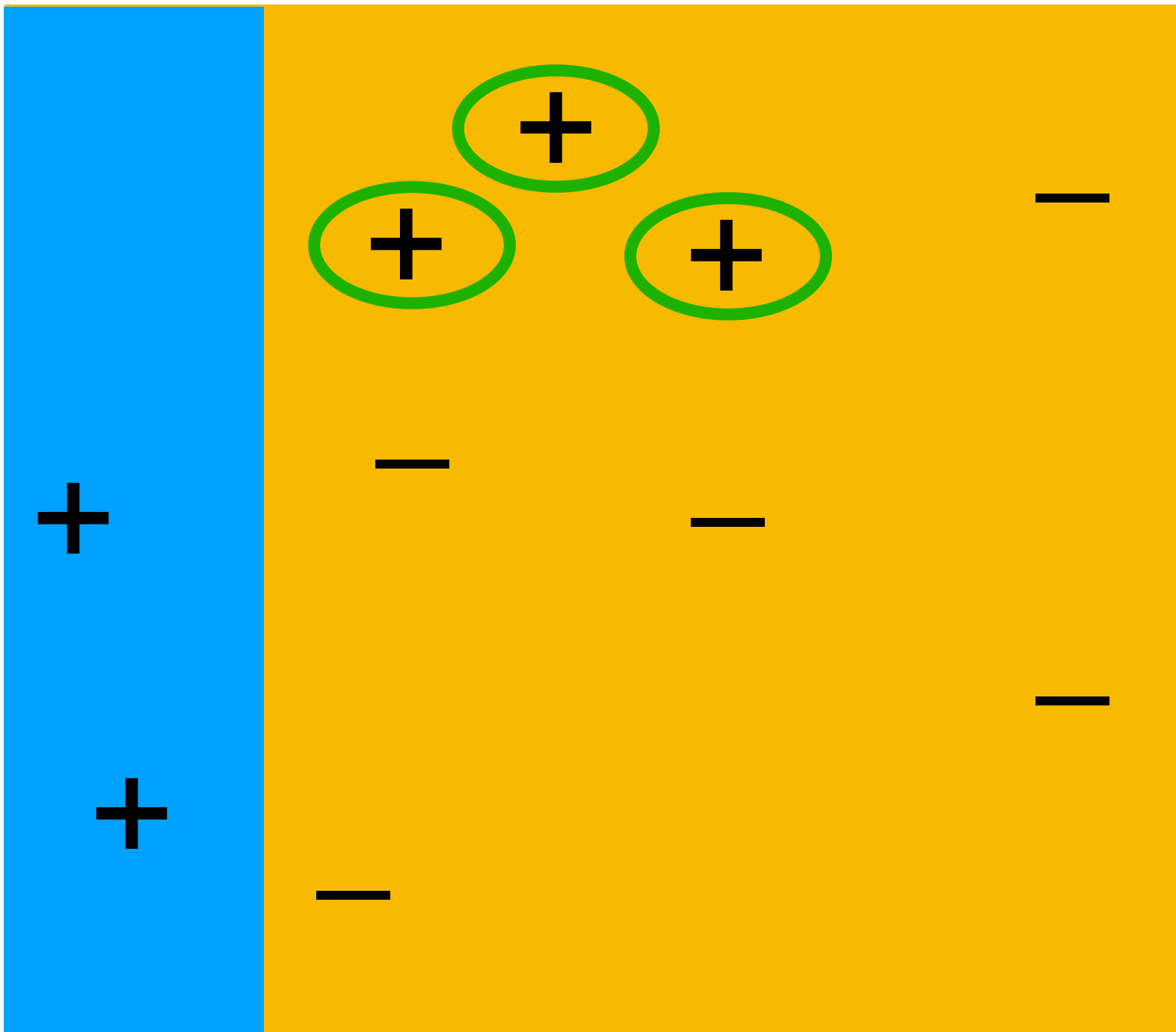
Round 1: Weak learner h_1
(misclassifies 3 items)

0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

Initially
 $1/n$

A BOOSTING ALGORITHM: AdaBoost



Round 1: Weak learner h_1
(misclassifies 3 items)

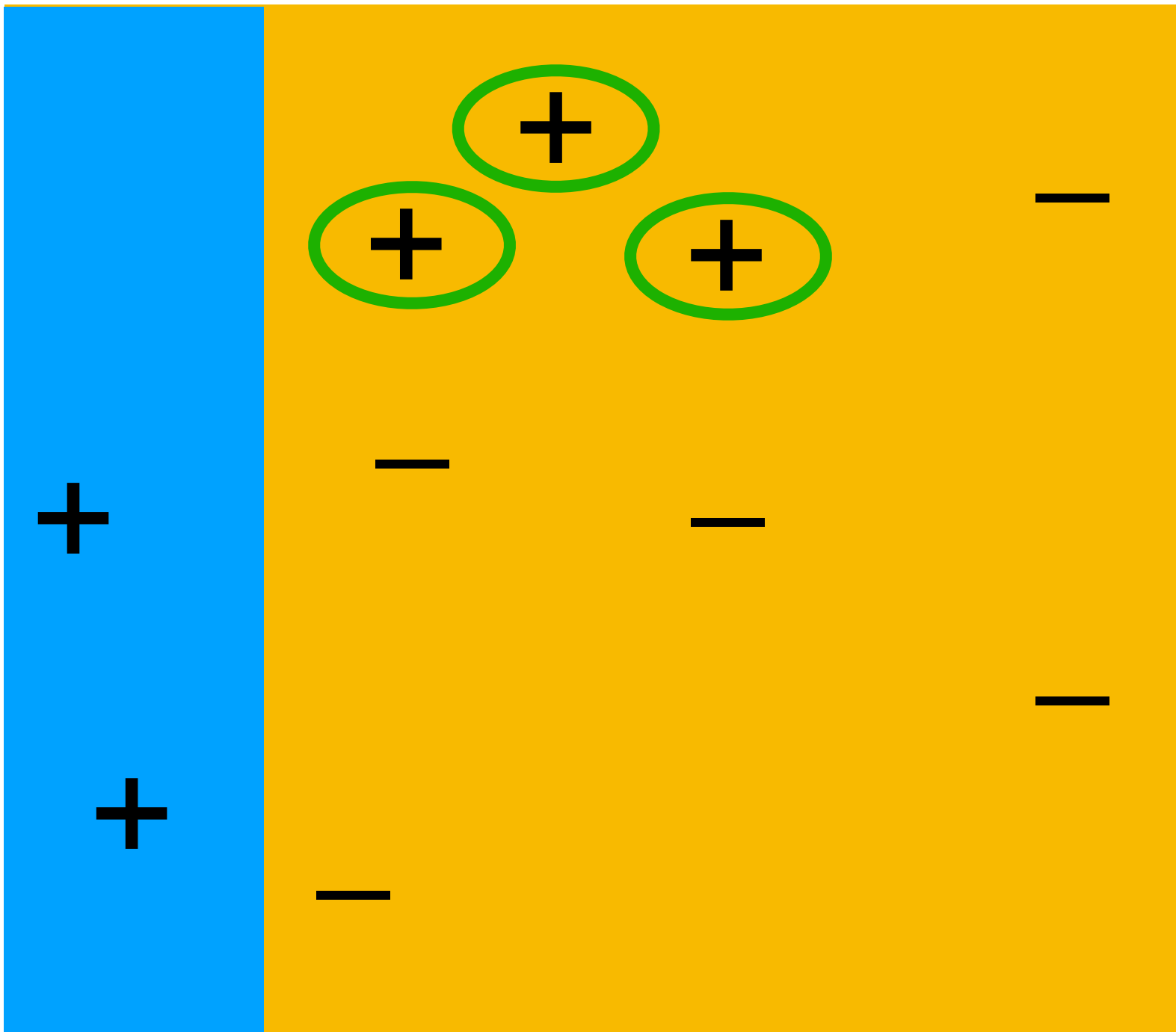
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

0
0
0
0
1
1
0
1
0
0

Errors

A BOOSTING ALGORITHM: AdaBoost



Round 1: Weak learner h_1

$\epsilon_1 = 0.3$
 $\alpha_1 = 1/2 \ln(\frac{1 - \epsilon_1}{\epsilon_1}) = 0.42$

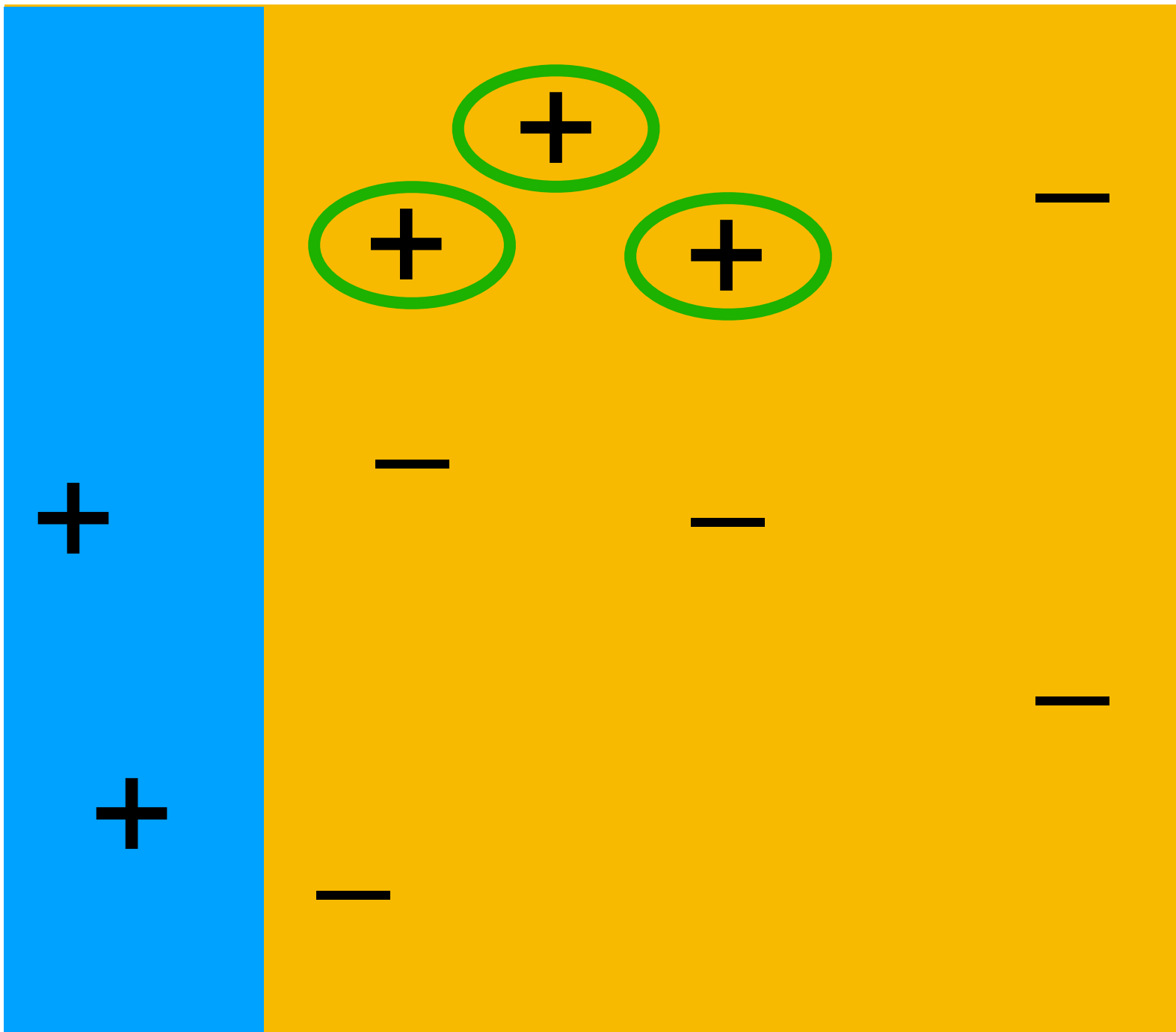
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

0	0.00
0	0.00
0	0.00
0	0.00
1	0.10
1	0.10
0	0.00
1	0.10
0	0.00
0	0.00

Errors
Weighted errors

A BOOSTING ALGORITHM: AdaBoost



0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

Round 1: Weak learner h_1

w_{i0}

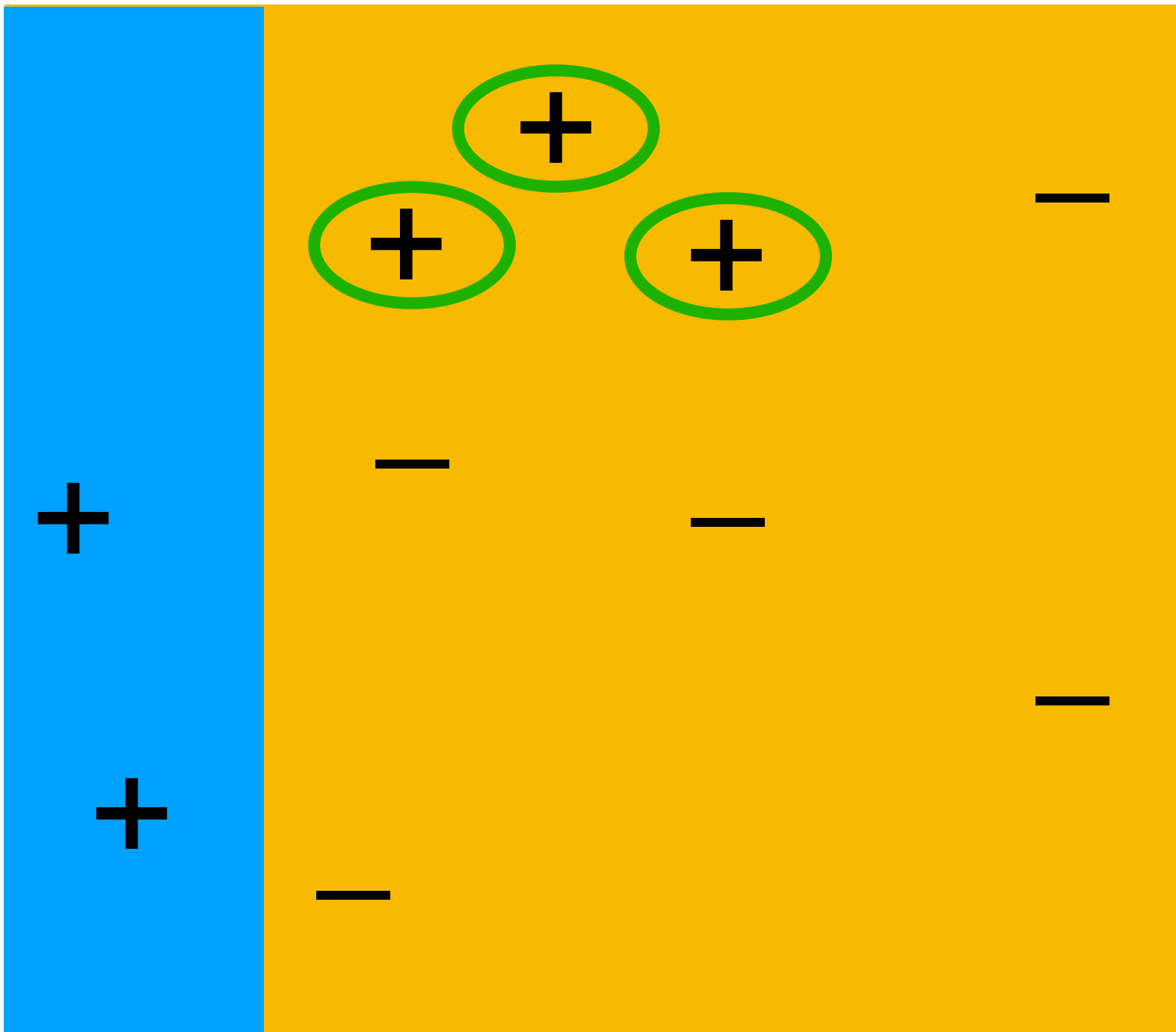
$\epsilon_1 = 0.3$
 $\alpha_1 = 1/2 \ln(\frac{1 - \epsilon_1}{\epsilon_1}) = 0.42$

Update Weights

$w_{i1} = 0.1 \times e^{-0.42}$ for correct
 $w_{i1} = 0.1 \times e^{0.42}$ for incorrect

} Normalize

A BOOSTING ALGORITHM: AdaBoost



Round 1: Weak learner h_1

$$\epsilon_1 = 0.3$$
$$\alpha_1 = 1/2 \ln\left(\frac{1 - \epsilon_1}{\epsilon_1}\right) = 0.42$$

0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10
0.10

w_{i0}

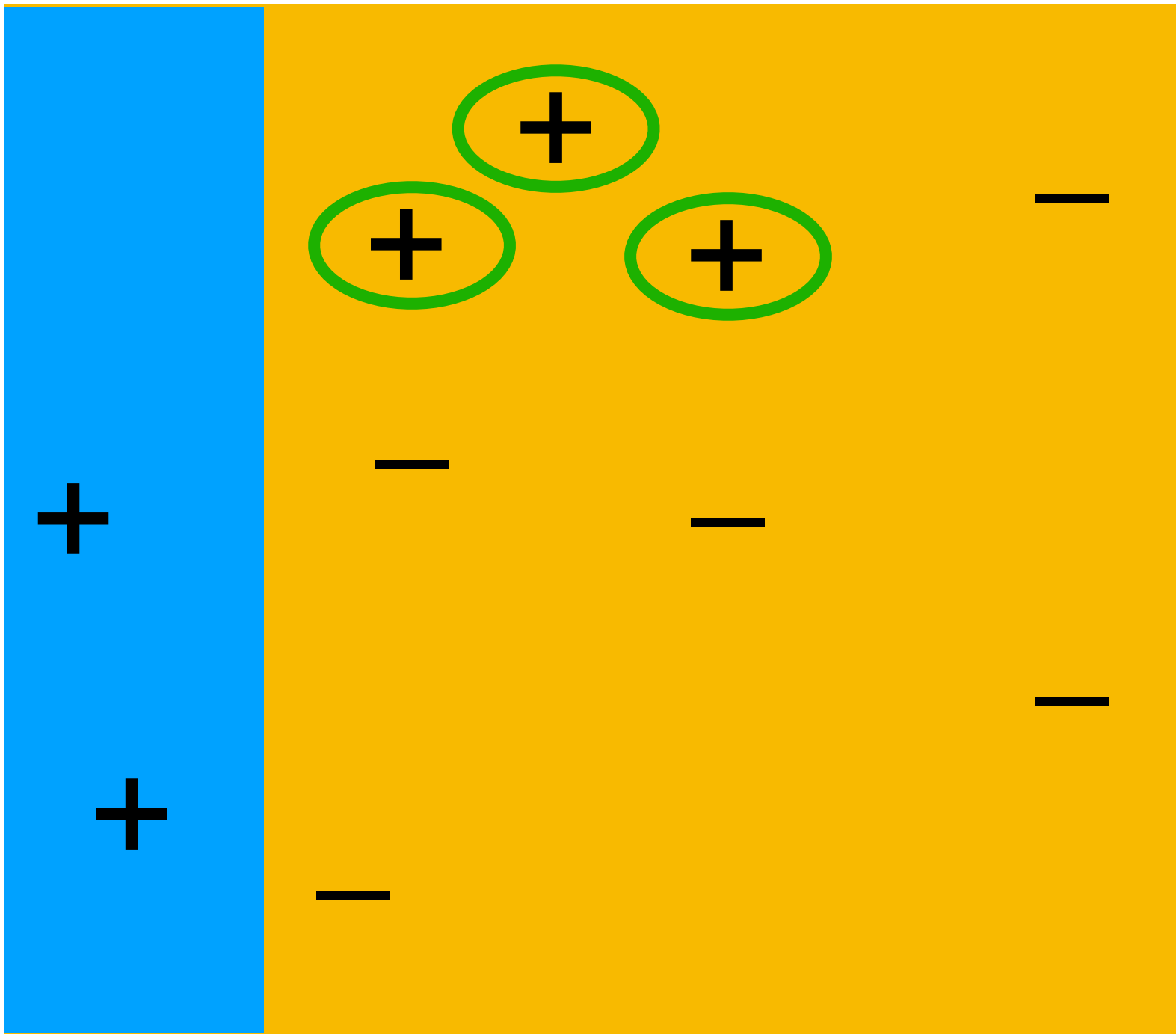
0.07
0.07
0.07
0.07
0.17
0.17
0.07
0.17
0.07
0.07

w_{i1}

Update Weights

$$\left. \begin{aligned} w_{i1} &= 0.1 \times e^{-0.42} \text{ for correct} \\ w_{i1} &= 0.1 \times e^{0.42} \text{ for incorrect} \end{aligned} \right\} \text{Normalize}$$

A BOOSTING ALGORITHM: AdaBoost

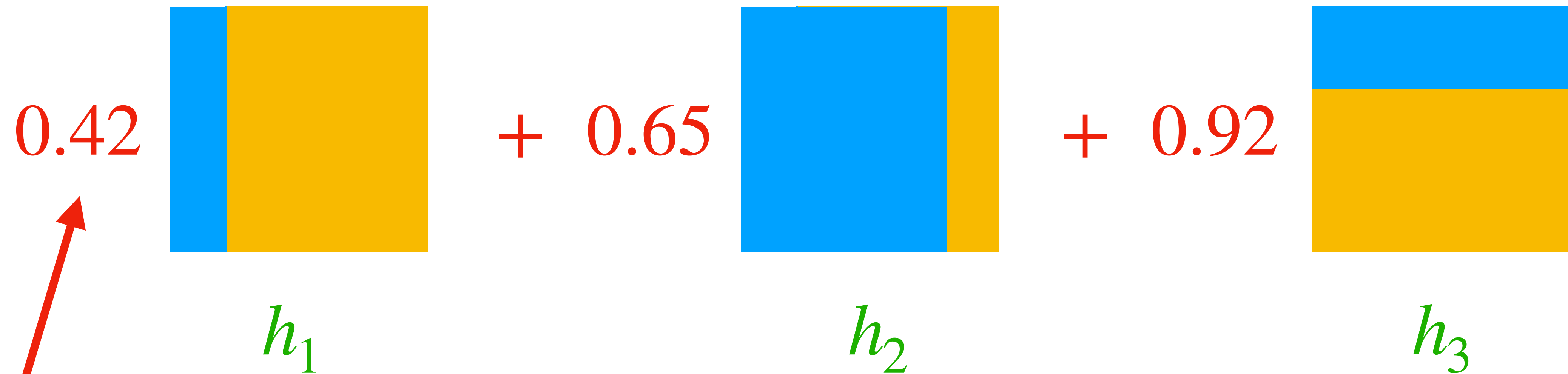


Round 1: Weak learner h_1



Misclassified points have **boosted weights**

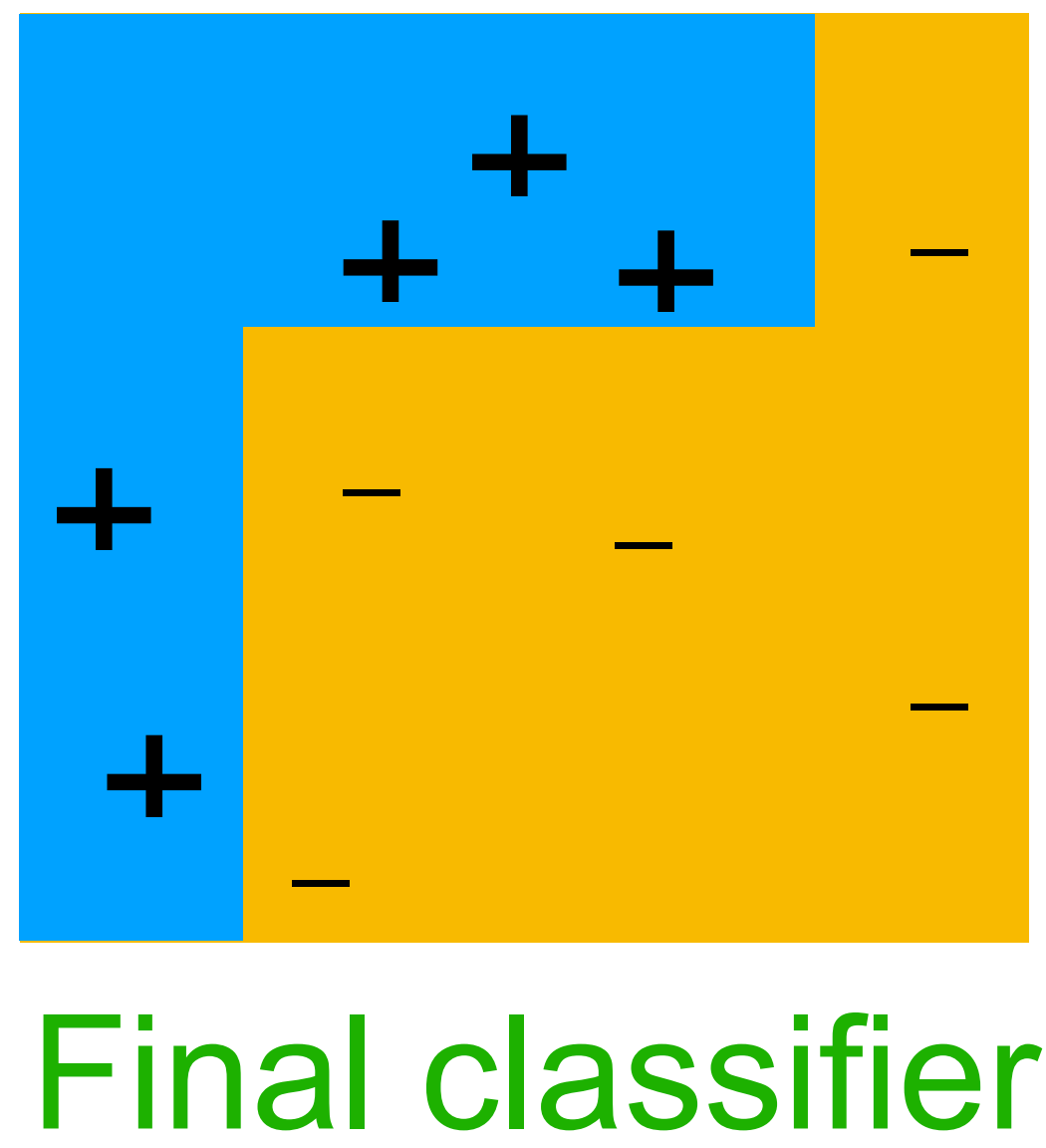
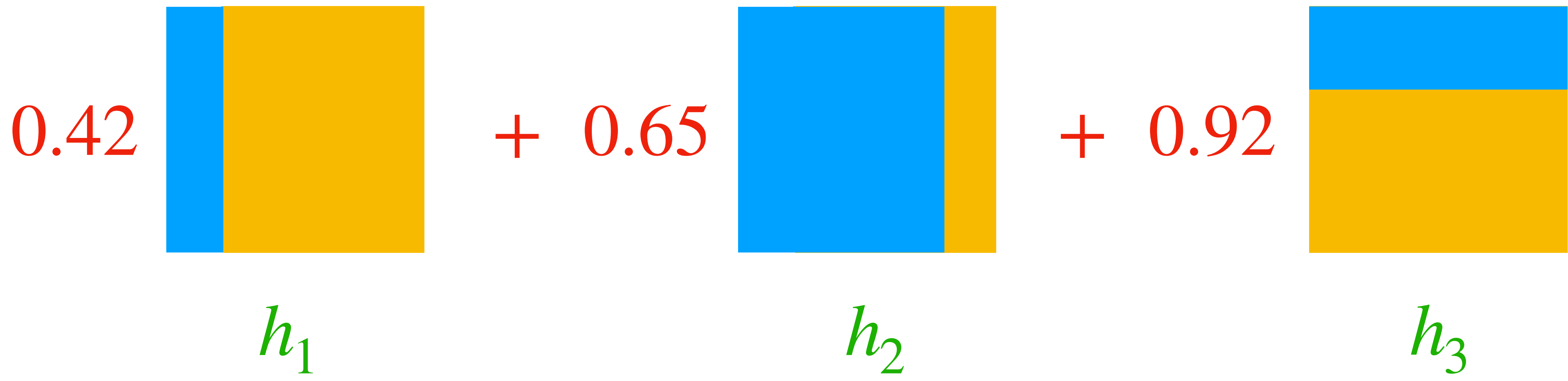
A BOOSTING ALGORITHM: AdaBoost



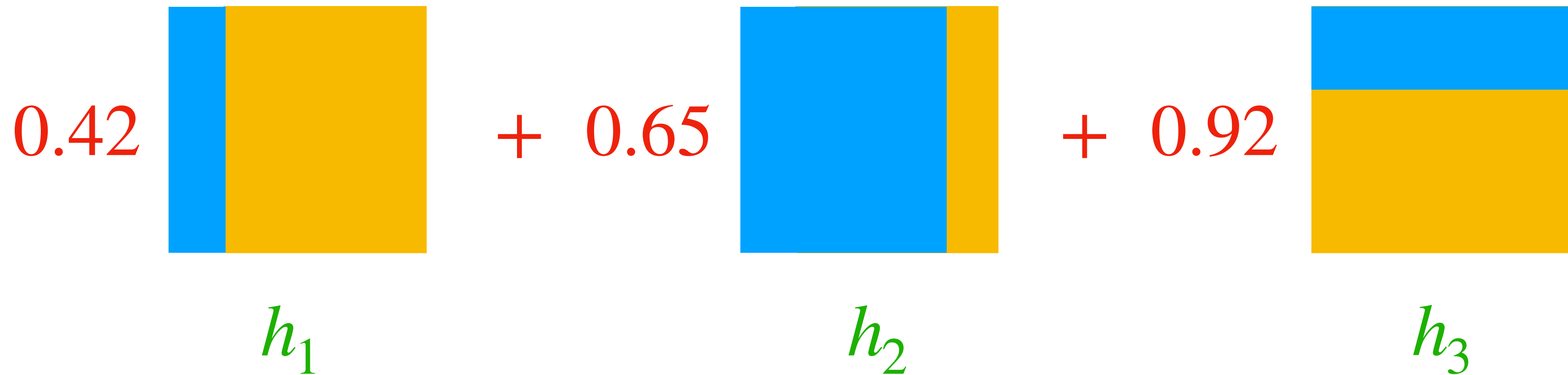
Weight of h_1 .

We computed it in the previous slides. Others are computed similarly.

A BOOSTING ALGORITHM: AdaBoost



A BOOSTING ALGORITHM: AdaBoost



To classify a new test point check the sign of

$$0.42(h_1 ? 1 \mid - 1) + 0.65(h_2 ? 1 \mid - 1) + 0.92(h_3 ? 1 \mid - 1)$$

MORE CLASSIFICATION ALGORITHMS

So far considered logistic regression, SVM, and decision trees.

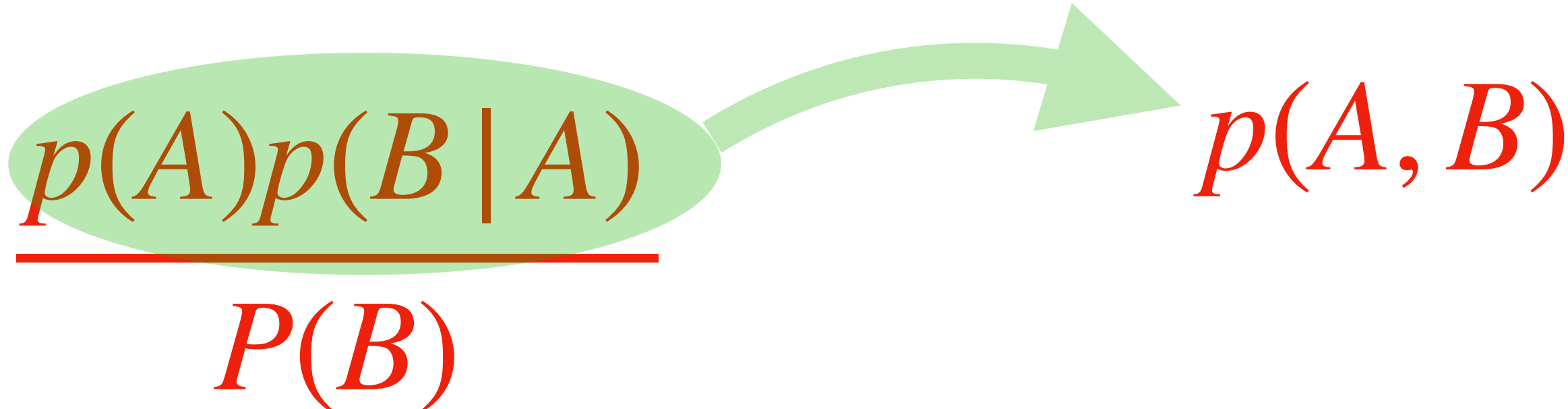
Now a new classification algorithm: Naive Bayes

MORE CLASSIFICATION ALGORITHMS

So far considered logistic regression, SVM, and decision trees.

Now a new classification algorithm: Naive Bayes

Recall: $p(A | B) = \frac{p(A)p(B | A)}{P(B)}$



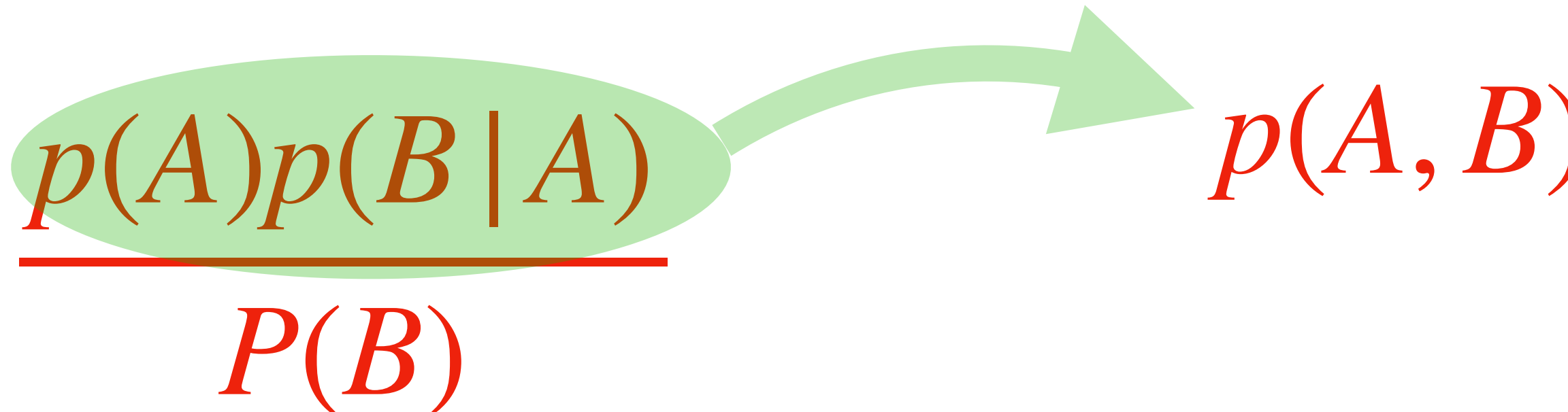
$p(A, B)$

MORE CLASSIFICATION ALGORITHMS

So far considered logistic regression, SVM, and decision trees.

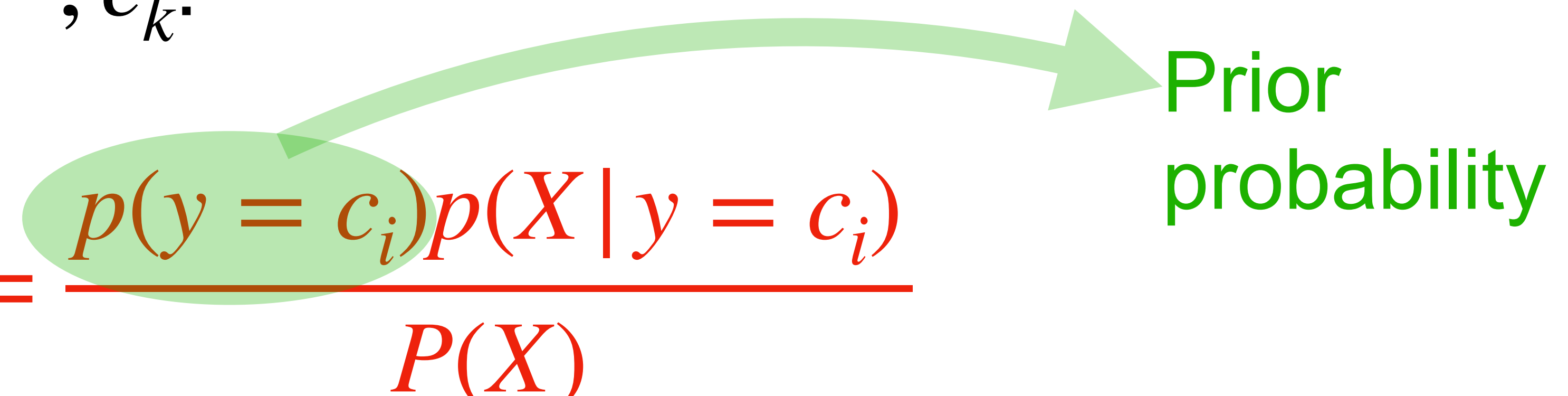
Now a new classification algorithm: Naive Bayes

Recall: $p(A | B) = \frac{p(A)p(B | A)}{P(B)}$



$p(A, B)$

Applying this to the framework where we have a set of training points with known classes $c_1, c_2, \dots, c_k$:

$$p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{P(X)}$$


Prior probability

NAIVE BAYES CLASSIFIER

Select class with max probability for item X :

$$\mathit{arg} \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)}]$$

NAIVE BAYES CLASSIFIER

Select class with max probability for item X :

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)}]$$

Denominator same for all classes: $p(y = c_i | X) \propto p(y = c_i)p(X | y = c_i)$

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i)p(X | y = c_i)]$$

NAIVE BAYES CLASSIFIER

Select class with max probability for item X :

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i | X) = \frac{p(y = c_i)p(X | y = c_i)}{p(X)}]$$

Denominator same for all classes: $p(y = c_i | X) \propto p(y = c_i)p(X | y = c_i)$

$$\arg \max_{c_i \in \{c_1, \dots, c_k\}} [p(y = c_i)p(X | y = c_i)]$$

To compute easily assume features independent given target class:
(Naive Bayes)

$$p(X | y = c_i) = p(x_1, x_2, \dots, x_m | y = c_i) = \prod_{j=1}^m p(x_j | y = c_i)$$